Chapter 6

DPO — Direct Preference Optimization

6.1 Motivation

PPO requires 4 models in memory (policy, reference, reward model, value head), complex RL infrastructure, and is notoriously unstable. DPO [302] asks: *can we skip the RL and learn directly from preferences?*

Key insight: The optimal policy under the RLHF objective (reward maximization + KL penalty) has a **closed-form solution**. We can derive a supervised loss that implicitly optimizes the same objective.

6.2 Mathematical Derivation

Step 1: RLHF objective: $\max_\pi \mathbb{E}_{x,y\sim\pi}[r(x,y)] - \beta D_{\mathrm{KL}}[\pi\|\pi_{\mathrm{ref}}]$

Step 2: The optimal solution is: $\pi^*(y|x) = \frac{1}{Z(x)}\pi_{\mathrm{ref}}(y|x)\exp\left(\frac{r(x,y)}{\beta}\right)$

Step 3: Rearrange to express reward in terms of policy: $r(x,y) = \beta\log\frac{\pi^*(y|x)}{\pi_{\mathrm{ref}}(y|x)} + \beta\log Z(x)$

Step 4: Substitute into Bradley-Terry preference model $P(y_w \succ y_l) = \sigma(r(y_w) - r(y_l))$. The $Z(x)$ cancels!

$$\boxed{\mathcal{L}_{\mathrm{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\mathrm{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\mathrm{ref}}(y_l|x)}\right)\right]} \tag{6.1}$$

What DPO Actually Does

Define the **implicit reward** as $\hat{r}(x,y) = \beta\log\frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)}$.
DPO minimizes the cross-entropy loss where the "label" is: chosen should have higher implicit reward than rejected. The margin is controlled by β:

- Large β: need large margin → policy moves aggressively → risk forgetting
- Small β: small margin suffices → policy stays close to reference → conservative

The reference model acts as a regularizer: the policy must "justify" any deviation from it by showing preference alignment.

6.3 Gradient Analysis

The DPO gradient decomposes as:

$$\nabla_\theta\mathcal{L} = -\beta\cdot \underbrace{\sigma(-\hat{r}_w + \hat{r}_l)}_{\text{weight: higher when model is wrong}} \cdot[\nabla_\theta\log\pi_\theta(y_w|x) - \nabla_\theta\log\pi_\theta(y_l|x)] \tag{6.2}$$

Interpretation: The gradient increases probability of chosen and decreases rejected. The weight is largest when the model currently prefers the wrong answer — it focuses learning on "confusing" pairs.

Concrete DPO Example

Prompt: "Explain quantum entanglement to a 10-year-old."
Chosen (y_w): "Imagine you have two magic coins. When you flip one and it's heads, the other one instantly becomes tails, no matter how far apart they are!"

$\log \pi_\theta(y_w|x) = -15.3$, $\log \pi_{\text{ref}}(y_w|x) = -16.1$
Rejected (y_l): "Quantum entanglement is a phenomenon where two particles become correlated such that the quantum state of one particle cannot be described independently."

$\log \pi_\theta(y_l|x) = -12.8$, $\log \pi_{\text{ref}}(y_l|x) = -12.5$
Implicit rewards: $\hat{r}_w = 0.1 \times ((-15.3) - (-16.1)) = 0.08$, $\hat{r}_l = 0.1 \times ((-12.8) - (-12.5)) = -0.03$
Loss input: $\sigma(0.08 - (-0.03)) = \sigma(0.11) = 0.527$
Loss: $-\log(0.527) = 0.64$ — The model barely prefers the chosen. Gradient will push hard.
After training: chosen probability increases, rejected decreases, until margin stabilizes around $1/(2\beta)$.

6.4 TRL Implementation

The following shows a minimal working example using HuggingFace TRL.

```
from trl import DPOConfig, DPOTrainer
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig
from datasets import load_dataset

model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-3.1-8B-Instruct",
    torch_dtype=torch.bfloat16, attn_implementation="flash_attention_2")
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")

# Dataset format: {"prompt": str, "chosen": str, "rejected": str}
dataset = load_dataset("argilla/ultrafeedback-binarized-preferences")

lora_config = LoraConfig(r=64, lora_alpha=16, lora_dropout=0.05,
    target_modules=["q_proj","k_proj","v_proj","o_proj","gate_proj","up_proj","
    down_proj"])

dpo_config = DPOConfig(
    output_dir="./dpo_output",
    beta=0.1,                    # KL regularization strength
    learning_rate=5e-7,          # Very low LR for stability
    loss_type="sigmoid",         # Standard DPO loss
    max_length=2048,             # Max sequence length
    max_prompt_length=1024,      # Truncation for prompts
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,  # Effective batch = 16
    gradient_checkpointing=True,
    bf16=True,
    num_train_epochs=1,          # DPO overfits fast - 1 epoch!
    warmup_ratio=0.1,
    logging_steps=10,
    eval_strategy="steps",
    eval_steps=200,
    save_strategy="steps",
    save_steps=500,
)

trainer = DPOTrainer(
    model=model,
```

```
    ref_model=None,              # With LoRA, ref = base model (no copy needed!)
    args=dpo_config,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    tokenizer=tokenizer,
    peft_config=lora_config,
)
trainer.train()
# Key metrics to monitor: train/rewards/chosen, train/rewards/rejected, train/
    rewards/margins
```

6.5 How DPO Works: Full Mechanics

This section provides the complete computational details of DPO — what happens at the token level during training.

6.5.1 Sequence-Level Log-Probabilities

The key quantity in DPO is the log-probability of an **entire sequence** $y = (y_1, y_2, \ldots, y_T)$ given prompt x. This is computed as the **sum of per-token log-probabilities**:

$$\boxed{\log \pi_\theta(y|x) = \sum_{t=1}^{T} \log \pi_\theta(y_t \mid x, y_{<t})} \tag{6.3}$$

Each term $\log \pi_\theta(y_t|x, y_{<t})$ is the log-softmax output at position t for the *actual* token y_t in the sequence. This is identical to the cross-entropy loss used in standard language modeling — but here we **sum** rather than average.

Critical detail: The gradient flows through **every token position** in both y_w and y_l. There is no masking of intermediate tokens — every token contributes to the sequence-level log-probability.

6.5.2 The DPO Loss Decomposed

Starting from the loss:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}[\log \sigma(\beta \cdot h_\theta(x, y_w, y_l))] \tag{6.4}$$

where the "implicit reward margin" h_θ is:

$$h_\theta(x, y_w, y_l) = \underbrace{\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)}}_{\text{chosen reward proxy}} - \underbrace{\log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}}_{\text{rejected reward proxy}} \tag{6.5}$$

Expanding into token-level terms:

$$\boxed{h_\theta = \sum_{t=1}^{|y_w|}\Big[\log \pi_\theta(y_w^t|x, y_w^{<t}) - \log \pi_{\text{ref}}(y_w^t|x, y_w^{<t})\Big] - \sum_{t=1}^{|y_l|}\Big[\log \pi_\theta(y_l^t|x, y_l^{<t}) - \log \pi_{\text{ref}}(y_l^t|x, y_l^{<t})\Big]} \tag{6.6}$$

6.5.3 Forward Pass: Step by Step

For one training example (x, y_w, y_l):

1. **Concatenate**: Form two sequences: $[x; y_w]$ and $[x; y_l]$. Pad to equal length within the batch.
2. **Forward pass (policy π_θ)**: Run both sequences through the model. Collect logits at every response position.

3. **Extract log-probs**: At each position t in the response, take $\log \text{softmax}(\text{logits}_t)[y_t]$ — the log-probability of the actual token.

4. **Sum over tokens**:

$$\text{logp\_chosen} = \sum_{t \in \text{response positions}} \log \pi_\theta(y_w^t | x, y_w^{<t}) \tag{6.7}$$

$$\text{logp\_rejected} = \sum_{t \in \text{response positions}} \log \pi_\theta(y_l^t | x, y_l^{<t}) \tag{6.8}$$

5. **Subtract reference** (pre-computed or from second forward pass):

$$\text{ratio\_w} = \text{logp\_chosen} - \text{ref\_logp\_chosen} \tag{6.9}$$

$$\text{ratio\_l} = \text{logp\_rejected} - \text{ref\_logp\_rejected} \tag{6.10}$$

6. **Compute loss**: $\mathcal{L} = -\log \sigma(\beta \cdot (\text{ratio\_w} - \text{ratio\_l}))$

7. **Backward pass**: Gradients flow back through steps $5 \to 4 \to 3 \to 2$ to update θ.

6.5.4 Token-Level Gradient Analysis

Does every token get a gradient? Yes. The gradient with respect to the logits at position t in the chosen sequence is:

$$\frac{\partial \mathcal{L}}{\partial \text{logits}_t^{(w)}} = -\underbrace{\sigma(-\beta \cdot h_\theta)}_{\text{scaling factor}} \cdot \beta \cdot \frac{\partial \log \pi_\theta(y_w^t | \cdot)}{\partial \text{logits}_t^{(w)}} \tag{6.11}$$

Key insight: The scaling factor $\sigma(-\beta \cdot h_\theta)$ is **shared across all tokens** in both sequences. It acts as an adaptive learning rate:

- When h_θ is small (model can't distinguish chosen from rejected): scaling ≈ 0.5 — strong gradient, learn aggressively.
- When h_θ is large (model already prefers chosen): scaling ≈ 0 — negligible gradient, don't over-fit.

Effect on chosen tokens: Probability is *increased* (log-prob pushed up).

Effect on rejected tokens: Probability is *decreased* (log-prob pushed down).

Relative to reference: Only the *difference* from π_{ref} matters. If the model already assigns high probability to the chosen response (matching the reference), there's little gradient.

6.5.5 Per-Token vs. Sequence-Level: Length Normalization

A subtle issue: longer sequences naturally have lower log-probabilities (more terms summed, each ≤ 0). If $|y_w| \gg |y_l|$, the loss can be biased toward preferring shorter responses.

Solutions:

- **Length-normalized DPO**: Replace $\log \pi_\theta(y|x)$ with $\frac{1}{|y|} \sum_t \log \pi_\theta(y_t|\cdot)$. Used in some implementations (SimPO adopts this).
- **Standard DPO**: Uses raw sum (no normalization). This *implicitly* penalizes verbosity — the model must assign high probability to every token in the chosen response.
- **Practical impact**: On benchmarks, length-normalized DPO reduces length gaming but can hurt instruction-following quality. Standard (unnormalized) is more common in production.

6.5.6 Label Masking: What Gets Gradients

Which Tokens Receive Gradient in DPO

- **Prompt tokens** (x): **NO gradient**. The loss is computed only over response positions. Prompt tokens provide context but their logits don't contribute to $\log \pi(y|x)$.
- **Chosen response tokens** (y_w): **ALL tokens get gradient**. Each y_w^t contributes to the sum. Gradient pushes their probabilities up.
- **Rejected response tokens** (y_l): **ALL tokens get gradient**. Each y_l^t contributes to the sum. Gradient pushes their probabilities down.
- **Padding tokens**: **NO gradient**. Masked out with attention mask.

6.5.7 Pseudocode: DPO Training Step

DPO Forward + Backward (PyTorch-style)

```
def dpo_loss(model, ref_model, batch, beta=0.1):
    """One DPO training step."""
    # batch contains: input_ids_chosen, input_ids_rejected,
    #                 labels_chosen, labels_rejected (prompt masked to -100)

    # 1. Forward pass: get per-token log-probs
    logps_chosen = get_sequence_logprob(model, batch["chosen"])
    logps_rejected = get_sequence_logprob(model, batch["rejected"])

    # 2. Reference log-probs (pre-computed or computed here)
    with torch.no_grad():
        ref_logps_chosen = get_sequence_logprob(ref_model, batch["chosen"])
        ref_logps_rejected = get_sequence_logprob(ref_model, batch["rejected"])

    # 3. Compute implicit reward margins
    chosen_rewards = beta * (logps_chosen - ref_logps_chosen)
    rejected_rewards = beta * (logps_rejected - ref_logps_rejected)

    # 4. DPO loss = -log(sigmoid(chosen_reward - rejected_reward))
    loss = -F.logsigmoid(chosen_rewards - rejected_rewards).mean()
    return loss

def get_sequence_logprob(model, sequences):
    """Sum of log-probs over response tokens only."""
    outputs = model(sequences["input_ids"], attention_mask=sequences["mask"])
    logits = outputs.logits[:, :-1, :]  # Shift for next-token prediction

    # Gather log-prob of actual tokens
    labels = sequences["labels"][:, 1:]  # Shifted labels
    log_probs = F.log_softmax(logits, dim=-1)
    token_logps = log_probs.gather(-1, labels.unsqueeze(-1)).squeeze(-1)

    # Mask: only sum over response tokens (labels != -100)
    mask = (labels != -100).float()
    return (token_logps * mask).sum(dim=-1)  # Shape: [batch_size]
```

6.5.8 Common Pitfalls

DPO Implementation Pitfalls

- **Forgetting to mask the prompt**: If prompt tokens are included in the log-prob sum, the model optimizes for prompt likelihood (useless) and the effective β is wrong.
- **Using mean instead of sum**: $\frac{1}{T}\sum_t \log \pi$ vs. $\sum_t \log \pi$ — these give different implicit length penalties. Must be consistent between π_θ and π_{ref}.

- **Stale reference model**: If π_{ref} is too far from π_θ (e.g., base model vs. fine-tuned), the KL term dominates and gradients vanish. Solution: use the SFT checkpoint (not base) as reference.
- β **too large**: Magnifies log-prob differences → sigmoid saturates → zero gradients. Start with $\beta = 0.1$, tune in $[0.05, 0.5]$.
- β **too small**: Theoretically allows more freedom from reference (weaker KL constraint), but the gradient $\propto \beta \cdot \sigma(-\beta h)$ becomes vanishingly small → loss landscape is flat → extremely slow convergence. The model has "permission" to move far but receives almost no signal telling it *where* to move.

6.6 DPO Variants and When Each Fails

When DPO Fails

1. Distribution shift: Preference data from old model. Current policy generates different text → loss is optimizing on irrelevant examples.
2. No exploration: Can't discover behaviors not in dataset. Stuck in local optimum.
3. Reference collapse: If reference is too strong, policy can't move. If too weak, no regularization.
4. Data quality: Noisy labels poison training. Unlike PPO which averages over many samples, DPO memorizes individual pairs.
5. Preference data diversity: Ensure chosen/rejected pairs cover the full spectrum of quality differences (not just good-vs-terrible). Pairs that differ in *approach*, not just quality, teach richer policy distinctions.

6.7 β Selection Guide

β	Regime	When to Use
0.01	Very aggressive	Only if data is extremely clean and you need large distributional shift
0.05	Aggressive	Good data, want noticeable improvement over SFT
0.1	Standard	Default starting point. Good balance of quality vs stability
0.2	Conservative	Noisy data, or model already close to desired behavior
0.5	Very conservative	Safety fine-tuning where you must not break capabilities

6.8 DPO Batch Size Configuration and Scaling

Unlike standard SFT which operates on single-sequence token predictions, DPO leverages a **pairwise loss** comparing a preferred sequence against a dispreferred sequence. This fundamentally alters memory utilization and optimization stability.

6.8.1 Global Batch Size Target

Empirical evidence across DPO implementations establishes an optimal global batch size range:

$$\boxed{B_{\text{global}} \in [32, 128]} \tag{6.12}$$

- $B_{\text{global}} < 32$: Severe gradient noise in implicit reward estimation → policy oscillates destructively between alignment goals (helpfulness vs. safety).

- $B_{\text{global}} > 128$: Diminishing returns on convergence velocity; high communication overhead across distributed compute.

6.8.2 Mathematical Decomposition

Because DPO loads **two** model copies simultaneously (active policy π_θ + frozen reference π_{ref}), per-sequence memory is doubled. The global batch size is decomposed as:

$$\boxed{B_{\text{global}} = B_{\text{micro}} \times N_{\text{GPUs}} \times K_{\text{accum}}} \tag{6.13}$$

- B_{micro}: Per-device micro-batch size (preference pairs per forward pass).
- N_{GPUs}: Number of parallel data-processing devices.
- K_{accum}: Gradient accumulation steps before weight update.

The pairing multiplier: A single DPO data instance contains a prompt (x), chosen (y_w), and rejected (y_l). The actual tensor load per micro-batch:

$$T_{\text{sequences}} = 2 \times B_{\text{micro}} \tag{6.14}$$

For models >7B parameters on 80GB GPUs with context lengths 4096–8192 tokens, the physical limit is rigidly constrained to $B_{\text{micro}} \in [1, 2]$.

6.8.3 Distributed Scaling Configurations

Table 6.1: Distributed scaling profiles for DPO training ($B_{\text{global}} = 64$ target).

Configuration	Single GPU	8-GPU Node
B_{global}	64	64
B_{micro}	2 (4 sequences)	2 (4 sequences)
N_{GPUs}	1	8
K_{accum}	32 steps	4 steps
Throughput	Sequential/slow	High parallel throughput

6.8.4 VRAM Optimization: Pre-computing Reference Log-Probabilities

The DPO loss:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right] \tag{6.15}$$

Because π_{ref} is **completely static** throughout training, its outputs can be pre-computed:

Reference Model Eviction Strategy

1. Execute a forward pass over dataset $\mathcal{D}$ using only π_{ref} before training begins.
2. Cache the scalars $\log \pi_{\text{ref}}(y_w|x)$ and $\log \pi_{\text{ref}}(y_l|x)$ to disk.
3. **Evict π_{ref} completely from GPU memory.**

Result: Available GPU memory doubles → can increase B_{micro} from 1–2 to 4–8, maximizing hardware utilization and training throughput.
Implementation: In TRL, set `precompute_ref_log_probs=True` in `DPOConfig`. For 70B models, this saves ∼140GB of GPU memory across the cluster.

6.9 DPO Extensions and Variants

Direct Preference Optimization (DPO) reformulates RLHF as a supervised learning problem by deriving a closed-form mapping between the reward function and the optimal policy. The standard DPO loss is:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(q,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|q)}{\pi_{\text{ref}}(y_w|q)} - \beta \log \frac{\pi_\theta(y_l|q)}{\pi_{\text{ref}}(y_l|q)}\right)\right],$$

where y_w is the preferred (winning) response, y_l is the dispreferred (losing) response, and β controls the strength of the KL penalty. The following subsections cover the most important extensions and variants.

6.9.1 f-DPO – Generalised f-Divergence DPO

Beyond Reverse KL

Standard DPO uses reverse KL divergence as the regulariser between policy and reference. Reverse KL is *mode-seeking*: it prefers to concentrate probability mass on a few high-reward responses. Forward KL is *mode-covering*: it spreads probability mass to cover all plausible responses. f-DPO [363] generalises to any f-divergence, allowing practitioners to trade off these behaviours.

The f-DPO loss replaces the log-ratio with the derivative of the f-divergence generator:

$$\mathcal{L}_{f\text{-DPO}} = -\mathbb{E}\left[f'\left(\frac{\pi_\theta(y_w|q)}{\pi_{\text{ref}}(y_w|q)}\right) - f'\left(\frac{\pi_\theta(y_l|q)}{\pi_{\text{ref}}(y_l|q)}\right)\right],$$

where f' is the derivative of the f-divergence generator function.

f-Divergence Options in TRL

- **reverse_kl**: $f'(u) = \log u$. Standard DPO. Mode-seeking.
- **forward_kl**: $f'(u) = -1/u$. Mode-covering. Better diversity.
- **js_divergence**: $f'(u) = \log(2u/(u+1))$. Balanced mode-seeking/covering.
- **alpha_divergence**: $f'(u) = u^{\alpha-1}$. Interpolates between forward and reverse KL.

f-DPO in TRL

```
from trl import DPOConfig, DPOTrainer

# Jensen-Shannon divergence (balanced)
config = DPOConfig(
    f_divergence_type="js_divergence",
    beta=0.1,
)

# Alpha divergence (alpha=0: forward KL, alpha=1: reverse KL)
config_alpha = DPOConfig(
    f_divergence_type="alpha_divergence",
    f_alpha_divergence_coef=0.5,   # alpha parameter
    beta=0.1,
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=config,
    train_dataset=dataset,
)
```

When to Use f-DPO

- Use **JS divergence** when you want a balance between diversity and quality.
- Use **forward KL** for creative tasks where diversity is paramount.
- Use **reverse KL** (standard DPO) for tasks with a single correct answer.
- Use **alpha divergence** to continuously interpolate and tune the trade-off.

6.9.2 Robust DPO

Noisy Labels in Preference Data

Human preference annotations are noisy. Annotators disagree, make mistakes, and sometimes flip the preferred/dispreferred labels. Standard DPO treats all labels as ground truth, which can cause the model to overfit to noise. Robust DPO [56] analytically debiases the loss under a known noise model.

Assume each label is flipped with probability ϵ (the noise rate). The debiased loss is:

$$\mathcal{L}_{\text{robust}} = \frac{(1-\epsilon)\,\mathcal{L}_{\text{DPO}}(y_w, y_l) - \epsilon\,\mathcal{L}_{\text{DPO}}(y_l, y_w)}{1-2\epsilon},$$

where $\mathcal{L}_{\text{DPO}}(y_w, y_l)$ is the standard DPO loss treating y_w as preferred, and $\mathcal{L}_{\text{DPO}}(y_l, y_w)$ is the loss with labels flipped. This correction removes the bias introduced by label noise.

Intuition for Robust DPO

The formula is a linear combination that "subtracts out" the contribution of flipped labels. When $\epsilon = 0$, it reduces to standard DPO. When $\epsilon = 0.5$, the denominator goes to zero – the labels are pure noise and no learning is possible. In practice, $\epsilon \in [0.05, 0.2]$ covers most real annotation noise levels.

Robust DPO in TRL

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
    loss_type="robust",
    label_smoothing=0.1,   # corresponds to epsilon = 0.1
    beta=0.1,
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=config,
    train_dataset=dataset,
)
```

6.9.3 TR-DPO – Trust Region DPO

Stale Reference Model Problem

Standard DPO uses a fixed reference model π_{ref} throughout training. As the policy π_θ improves, the KL penalty $\beta \log(\pi_\theta/\pi_{\text{ref}})$ grows, eventually dominating the loss and preventing further improvement. TR-DPO [116] periodically updates the reference model to track the current policy.

TR-DPO updates the reference model using an exponential moving average (EMA):

$$\pi_{\text{ref}}^{(t+1)} \leftarrow \alpha \cdot \pi_\theta^{(t)} + (1-\alpha) \cdot \pi_{\text{ref}}^{(t)},$$

where $\alpha \in (0, 1)$ is the mixup coefficient. This is applied every T_{sync} gradient steps.

TR-DPO in TRL

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
    loss_type="sigmoid",          # standard DPO loss
    sync_ref_model=True,          # enable TR-DPO
    ref_model_mixup_alpha=0.6,    # alpha: how much of current policy to mix in
    ref_model_sync_steps=512,     # T_sync: sync every 512 steps
    beta=0.1,
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=config,
    train_dataset=dataset,
)
```

When to Use TR-DPO

- Long training runs where the policy drifts far from the initial reference.
- When you observe the DPO loss plateauing early due to KL penalty domination.
- Iterative DPO pipelines where new preference data is collected from the current policy.
- Set α close to 1 for fast reference updates; close to 0 for slow updates.

6.9.4 EXO – Exact Optimisation

DPO's KL Direction Problem

DPO is derived by solving for the optimal policy under a reverse KL constraint. However, the resulting loss actually optimises a *forward* KL objective in the reward space, which is the wrong direction. EXO [163] corrects this by using reverse KL probability matching, which is the theoretically correct objective for alignment.

EXO minimises the reverse KL between the model distribution and the target (reward-optimal) distribution:

$$\mathcal{L}_{\text{EXO}} = \mathbb{E}_{y\sim\pi_\theta}\left[\log \frac{\pi_\theta(y|q)}{p^*(y|q)}\right],$$

where $p^*(y|q) \propto \pi_{\text{ref}}(y|q)\exp(r(y,q)/\beta)$ is the optimal policy. In practice, EXO approximates this using the available preference pairs:

$$\mathcal{L}_{\text{EXO}} \approx -\mathbb{E}\left[\log\sigma\left(\beta\log\frac{\pi_{\text{ref}}(y_w|q)}{\pi_\theta(y_w|q)} - \beta\log\frac{\pi_{\text{ref}}(y_l|q)}{\pi_\theta(y_l|q)}\right)\right].$$

Note the *swapped* roles of π_θ and π_{ref} compared to DPO.

EXO in TRL

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
    loss_type="exo_pair",
    beta=0.1,
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=config,
```

```
    train_dataset=dataset,
)
```

6.9.5 NCA – Noise Contrastive Alignment

Likelihood Collapse in DPO

A known failure mode of DPO is *likelihood collapse*: the model learns to decrease the probability of the losing response but also decreases the probability of the winning response (since the loss only cares about the *difference*). NCA [43] adds an absolute likelihood term to prevent this.

NCA reframes alignment as noise-contrastive estimation. The loss has three terms:

$$\boxed{\mathcal{L}_{\text{NCA}} = -\log \sigma(r_w) - \tfrac{1}{2}\log \sigma(-r_w) - \tfrac{1}{2}\log \sigma(-r_l),}$$

where $r_y = \beta \log(\pi_\theta(y|q)/\pi_{\text{ref}}(y|q))$ is the implicit reward. The first term encourages high reward for y_w; the second and third terms penalise high reward for both y_w and y_l (preventing collapse).

NCA in TRL

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
    loss_type="nca_pair",
    beta=0.01,    # small beta: absolute likelihood term dominates
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=config,
    train_dataset=dataset,
)
```

When to Use NCA

- When you observe the winning response probability decreasing during DPO training.
- For tasks where absolute response quality matters, not just relative ranking.
- Use small β (e.g., 0.01) to give the absolute likelihood term more weight.

6.9.6 SLiC-HF – Sequence Likelihood Calibration

Hinge Loss as a Simpler Alternative

The log-sigmoid loss in DPO is smooth but can be slow to converge when the margin is large. SLiC-HF [432] uses a hinge loss, which is zero when the margin exceeds a threshold and linear otherwise. This is simpler, faster, and surprisingly competitive.

The SLiC-HF loss is:

$$\mathcal{L}_{\text{SLiC}} = \max\left(0,\ \delta - \beta \log \frac{\pi_\theta(y_w|q)}{\pi_{\text{ref}}(y_w|q)} + \beta \log \frac{\pi_\theta(y_l|q)}{\pi_{\text{ref}}(y_l|q)}\right),$$

where δ is the margin threshold. When the model already assigns a margin of δ between winning and losing responses, the loss is zero.

SLiC-HF in TRL

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
```

```
    loss_type="hinge",
    beta=0.1,
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=config,
    train_dataset=dataset,
)
```

6.9.7 Iterative RPO – Reasoning Preference Optimisation

DPO Forgets How to Generate

Standard DPO trains the model to *discriminate* between winning and losing responses. But for reasoning tasks, the model also needs to *generate* correct reasoning traces. A model that can discriminate but not generate is useless at inference time. RPO adds an NLL (negative log-likelihood) term on the winning response to ensure the model learns to generate it.

The RPO loss combines DPO and SFT:

$$\mathcal{L}_{\text{RPO}} = \lambda_1 \mathcal{L}_{\text{DPO}}(y_w, y_l) + \lambda_2 \mathcal{L}_{\text{NLL}}(y_w),$$

where $\mathcal{L}_{\text{NLL}}(y_w) = -\log \pi_\theta(y_w|q)$ is the standard language modelling loss on the winning response.

Iterative RPO in TRL

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
    loss_type="sigmoid",            # Standard DPO loss
    rpo_alpha=1.0,                  # NLL regularisation weight (RPO)
    beta=0.1,
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=config,
    train_dataset=dataset,
)
```

When to Use RPO

- Reasoning tasks (math, code, logic) where the model must generate step-by-step solutions.
- When DPO training causes the model to lose fluency or generation quality.
- Iterative pipelines: generate rollouts, label them, train with RPO, repeat.
- The NLL term acts as a regulariser, preventing the policy from collapsing.

6.9.8 SimPO – Simple Preference Optimisation

Reference-Free Preference Learning

DPO requires a reference model to compute the implicit reward. This doubles memory usage and adds complexity. SimPO [252] eliminates the reference model by using the *average log-probability* of the response as an implicit reward, with a length normalisation term to prevent the model from preferring short responses.

SimPO defines the implicit reward as: